SIMATS ENGINEERING

ASSESSMENT TOOL 3: Mock Interview

COURSE NAME: Cloud Computing and Big Data Analytics **COURSE CODE:** CSA1522

Register Number	192521391
Name	Brijesh.T

COURSE OUTCOME COVERED

CO5: *Design big data processing systems using the Hadoop ecosystem including HDFS, MapReduce, and YARN.* (BL5)

Dave's Taxonomy: Articulation (Level 4) **Total Marks: 25**

Question Count: 10

Code of Conduct

I _Brijesh.T/192521391_ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: ____ _____

Assessment Tasks

Mock Interview

Answer the following interview-oriented questions clearly and concisely. Support your responses with architecture-level reasoning wherever appropriate.

1. Explain the difference between HDFS and traditional file systems.
2. Why is replication important in distributed storage systems?
3. How does MapReduce divide work between map and reduce phases?
4. What role does YARN play in large-scale Hadoop processing?
5. Differentiate NAS and SAN in an enterprise data environment.
6. What is the purpose of ETL in a big data pipeline?
7. How does Apache NiFi help in data integration workflows?
8. What are the key failure points in a Hadoop cluster and how would you handle them?
9. How would you design a secure data ingestion pipeline for a large organization?
10. How would you explain a scalable Hadoop-based processing system to a non-technical manager?

Mock Interview Rubric

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Technical Knowledge	25%	Shows deep and accurate technical understanding	Good technical understanding	Basic understanding	Weak understanding
Problem Solving	20%	Responds with strong architecture-level reasoning	Reasonable reasoning	Basic reasoning	Weak reasoning
Communication	20%	Answers are confident, precise, and professional	Mostly clear communication	Moderate clarity	Weak communication
Practical Awareness	20%	Connects concepts to real-world implementation well	Some practical relevance	Limited practical relevance	No practical linkage
Confidence and Professionalism	15%	Highly confident and professional	Generally professional	Moderately professional	Low confidence and professionalism

1. Difference between HDFS and traditional file systems

HDFS	Traditional File System
Designed for distributed storage	Usually designed for a single computer/server
Stores data across many machines	Stores data mainly on one machine
Can handle very large files	Better suited for smaller files
Provides data replication	Usually does not automatically replicate data
Can continue working even if a node fails	Failure of the main storage can cause data loss/unavailability
Used in Hadoop and big data applications	Used in normal computers and servers

In simple words: HDFS divides large data into blocks and stores them on different machines, making it suitable for big data.

2. Why is replication important in distributed storage?

Replication means keeping **multiple copies of the same data** on different machines.

It is important because:

- Protects data from hardware failure.
- Prevents data loss when a server goes down.
- Improves data availability.
- Allows users to access another copy when one node fails.
- Can improve read performance because data can be read from nearby copies.

For example, if a file has **3 replicas** and one server fails, the other two copies are still available.

3. How does MapReduce divide work between map and reduce phases?

MapReduce mainly has two phases:

Map Phase:

- Takes the input data.
- Divides the data into smaller parts.
- Processes each part.
- Produces intermediate **key-value pairs**.

Reduce Phase:

- Receives the intermediate key-value pairs.
- Groups similar keys together.
- Performs calculations or combines the values.
- Produces the final result.

Example: In counting words, the Map phase produces (word, 1), and the Reduce phase adds all the values for the same word.

Flow:
Input → Map → Shuffle/Sort → Reduce → Output

4. What role does YARN play in large-scale Hadoop processing?

YARN (Yet Another Resource Negotiator) is responsible for managing resources and jobs in a Hadoop cluster.

Its main functions are:

- Allocates CPU and memory to applications.
- Manages different jobs running at the same time.
- Monitors running applications.
- Decides where tasks should run.
- Improves resource utilization.
- Allows different types of applications to run on Hadoop.

In simple words: YARN acts like a **manager** that distributes available computer resources among different Hadoop jobs.

5. Difference between NAS and SAN

NAS	SAN
Network Attached Storage	Storage Area Network
Provides file-level storage	Provides block-level storage
Accessed using a network	Uses a dedicated storage network
Easier to install and manage	More complex to manage
Generally cheaper	Usually more expensive
Suitable for file sharing and backups	Suitable for databases and high-performance applications

Simple example: NAS is like a shared network folder, while SAN provides storage that servers can use almost like their own local disks.

6. Purpose of ETL in a big data pipeline

ETL means Extract, Transform, Load.

- **Extract:** Collect data from different sources.
- **Transform:** Clean, filter, format, and organize the data.
- **Load:** Put the processed data into a database, data warehouse, or storage system.

ETL helps organizations convert raw data into **clean and useful information** for analysis.

Example: Customer data from websites, mobile apps, and databases can be collected, cleaned, and loaded into a data warehouse.

7. How does Apache NiFi help in data integration?

Apache NiFi is a tool used to **move and process data between different systems**.

It helps by:

- Collecting data from different sources.
- Moving data between systems automatically.
- Filtering and transforming data.

- Monitoring data flows.
- Handling errors and failures.
- Supporting real-time data movement.
- Providing a visual interface for creating data flows.

In simple words: NiFi works like a **data traffic controller**, making sure data moves from its source to the correct destination.

8. Key failure points in a Hadoop cluster and how to handle them

Common failure points include:

1. **DataNode failure** – HDFS replication allows another copy of the data to be used.
2. **NameNode failure** – Use a highly available NameNode setup with a standby node.
3. **Network failure** – Use reliable and redundant network connections.
4. **Disk failure** – Use multiple disks and data replication.
5. **Resource overload** – Monitor CPU, memory, and storage and allocate resources properly.
6. **Application failure** – Configure job retry and monitoring mechanisms.
7. **Power failure** – Use backup power systems and proper recovery procedures.

Regular monitoring, replication, backups, and high-availability configurations help reduce the effect of failures.

9. Designing a secure data ingestion pipeline

A secure pipeline can be designed as:

Data Sources → Secure Transfer → Validation → Processing → Storage → Monitoring

Important security measures are:

- Use **encryption** while data is moving.
- Use encryption for stored sensitive data.
- Authenticate users and applications.
- Apply **role-based access control (RBAC)**.
- Validate incoming data to prevent malicious or incorrect data.
- Keep audit logs of data access and changes.
- Use firewalls and network security controls.
- Monitor the pipeline for unusual activity.
- Back up important data.
- Regularly update and patch the systems.

This ensures that only authorized users can access the data and that data remains protected throughout the pipeline.

10. Explaining a scalable Hadoop system to a non-technical manager

I would explain it using a simple example:

“Imagine a large warehouse containing millions of boxes. Instead of keeping all boxes in one huge room, we divide them among many smaller warehouses. If more boxes arrive, we can simply add more warehouses.”

Similarly, Hadoop:

- Divides large amounts of data into smaller pieces.
- Stores the pieces across multiple computers.
- Processes different pieces at the same time.

- Adds more computers when the amount of data increases.
- Continues working even if one computer fails.

In simple terms: Hadoop is scalable because we can **add more machines to handle more data and more workload**, instead of replacing one machine with a much bigger and more expensive machine.